

PolyFlop8

Instruction Set Reference Manual

Complete Instruction Descriptions and Operations

Revision: 2.0
Date: January 4, 2026

1 INSTRUCTION SET ARCHITECTURE OVERVIEW

1.1 Instruction Formats

The VHDL-MCU-8 uses three primary instruction formats:

Type	15:11	10:8	7:5	4:0
R-Type	Opcode (5)	Rd (3)	Rs (3)	Reserved (5)
I-Type	Opcode (5)	Rd (3)	Immediate (8)	
J-Type	Opcode (5)	Offset/Address (11)		

Table 1: Instruction Encoding Formats

1.2 Addressing Modes

- **Register Direct:** Operands are in registers (Rd, Rs)
- **Immediate:** 8-bit constant embedded in instruction
- **Direct Addressing:** Memory address specified in instruction
- **Indirect Addressing:** Memory address in register (X = R7)
- **Relative Addressing:** PC-relative branching

2 ARITHMETIC INSTRUCTIONS

2.1 ADD - Add Without Carry

Mnemonic	ADD Rd, Rs
Format	R-Type
Encoding	[Opcode][Rd][Rs][00000]
Operation	$Rd \leftarrow Rd + Rs$
Description	Adds the contents of source register Rs to destination register Rd. Stores the result in Rd. This is a standard 8-bit binary addition. Both operands are treated as unsigned integers, but the flags support both unsigned and signed interpretation.
Flags Affected	Z: Set if result = 0x00 C: Set if unsigned overflow (result > 0xFF) N: Set if bit 7 of result = 1 V: Set if signed overflow H: Set if carry from bit 3 to bit 4 S: N XOR V (for signed operations)
Cycles	1
Example	ADD R3, R4 ; R3 = R3 + R4

2.2 ADC - Add With Carry

Mnemonic	ADC Rd, Rs
Operation	$Rd \leftarrow Rd + Rs + C$
Description	Adds Rs, Rd, and the Carry flag. Used for multi-byte addition. If performing 16-bit addition with 8-bit registers, use ADD for low byte and ADC for high byte. The carry from the low byte addition propagates into the high byte addition.
Flags Affected	Same as ADD
Example	ADC R3, R4 ; R3 = R3 + R4 + Carry

2.3 SUB - Subtract Without Carry

Mnemonic	SUB Rd, Rs
Operation	$Rd \leftarrow Rd - Rs$
Description	Subtracts Rs from Rd using 2's complement arithmetic. C flag acts as "NOT borrow" - set if $Rd \geq Rs$ (unsigned), cleared if $Rd < Rs$. For signed subtraction, V flag indicates overflow.
Flags Affected	Z: Set if result = 0x00 C: Cleared if borrow needed ($Rd < Rs$) N: Set if result negative (bit 7 = 1) V: Set if signed overflow H: Cleared if borrow from bit 4
Example	SUB R5, R2 ; $R5 = R5 - R2$

2.4 SBC - Subtract With Carry

Mnemonic	SBC Rd, Rs
Operation	$Rd \leftarrow Rd - Rs - C$
Description	Subtracts Rs and the Carry flag from Rd. Used for multi-byte subtraction. If C=0, subtracts normally. If C=1, subtracts an additional 1 (propagates borrow from lower byte).
Flags Affected	Same as SUB
Example	SBC R3, R1 ; $R3 = R3 - R1 - \text{Carry}$

2.5 COM - One's Complement

Mnemonic	COM Rd
Format	Special R-Type (Rs field ignored)
Operation	$Rd \leftarrow 0xFF - Rd$
Description	Replaces Rd with its one's complement (bitwise NOT). Each 0 becomes 1, each 1 becomes 0. Equivalent to XOR with 0xFF. The C flag is always set.
Flags Affected	Z: Set if result = 0xFF C: Always set (1) N: Set if bit 7 = 1 V: Always cleared (0)
Example	COM R2 ; $R2 = \text{NOT } R2$ (bitwise)

2.6 NEG - Two's Complement Negation

Mnemonic	NEG Rd
Operation	$Rd \leftarrow 0x00 - Rd$
Description	Replaces Rd with its two's complement negative. Equivalent to $0x00 - Rd$. Note that NEG 0x80 gives 0x80 (-128 negated gives -128 due to overflow) and sets V flag.
Flags Affected	Same as SUB with Rs = Rd Special: NEG 0 sets Z=1, C=0
Example	NEG R4 ; R4 = -R4 (two's complement)

3 LOGICAL INSTRUCTIONS

3.1 AND - Logical AND

Mnemonic	AND Rd, Rs
Operation	$Rd \leftarrow Rd \wedge Rs$ (bitwise AND)
Description	Performs bitwise AND between Rd and Rs. Each bit of result is 1 only if both corresponding bits of operands are 1. Used for masking bits (clearing selected bits).
Flags Affected	Z: Set if result = 0x00 C: Cleared (0) N: Set if bit 7 = 1 V: Cleared (0)
Example	AND R3, R5 ; R3 = R3 AND R5 AND R2, 0x0F ; Mask upper nibble

3.2 OR - Logical OR

Mnemonic	OR Rd, Rs
Operation	$Rd \leftarrow Rd \vee Rs$ (bitwise OR)
Description	Performs bitwise OR between Rd and Rs. Each bit of result is 1 if either corresponding bit of operands is 1. Used for setting bits.
Flags Affected	Same as AND
Example	OR R1, R2 ; R1 = R1 OR R2 OR R3, 0x80 ; Set bit 7

3.3 XOR - Logical Exclusive OR

Mnemonic	XOR Rd, Rs
Operation	$Rd \leftarrow Rd \oplus Rs$ (bitwise XOR)
Description	Performs bitwise XOR. Each bit of result is 1 if corresponding bits differ. Useful for toggling bits, comparing values, and clearing registers (XOR with self = 0).
Flags Affected	Same as AND
Example	XOR R4, R4 ; Clear R4 (sets to 0) XOR R2, R3 ; R2 = R2 XOR R3

4 DATA TRANSFER INSTRUCTIONS

4.1 LDI - Load Immediate

Mnemonic	LDI Rd, k
Format	I-Type
Encoding	[Opcode][Rd][8-bit constant]
Operation	$Rd \leftarrow k$
Description	Loads an 8-bit immediate constant directly into register Rd. The constant k is embedded in the instruction itself (lower 8 bits). This is the primary way to load constants into registers.
Flags Affected	None
Cycles	1
Example	LDI R3, 0x42 ; R3 = 0x42 LDI R0, 255 ; R0 = 0xFF

4.2 MOV - Move Register

Mnemonic	MOV Rd, Rs
Operation	$Rd \leftarrow Rs$
Description	Copies the contents of source register Rs to destination register Rd. Both registers remain unchanged. Used for register-to-register transfers without affecting flags.
Flags Affected	None
Example	MOV R2, R3 ; Copy R3 to R2

4.3 LD - Load Indirect (X-pointer)

Mnemonic	LD Rd, X
Format	Special R-Type (X implied as R7)
Operation	$Rd \leftarrow (X)$ where $X = R7$
Description	Loads Rd from data memory location pointed to by X register (R7). Uses indirect addressing - the actual address is stored in R7. Post-increment/pre-decrement not supported.
Flags Affected	None
Cycles	2 (1 for fetch, 1 for memory read)
Example	LD R2, X ; R2 = memory[R7] LDI R7, 0x80 ; Set pointer LD R3, X ; R3 = memory[0x80]

4.4 ST - Store Indirect (X-pointer)

Mnemonic	ST X, Rs
Operation	$(X) \leftarrow Rs$ where $X = R7$
Description	Stores Rs to data memory location pointed to by X register (R7). The destination register in instruction is actually the source data.
Flags Affected	None
Cycles	2
Example	ST X, R3 ; memory[R7] = R3 LDI R7, 0x90 ; Set pointer ST X, R2 ; memory[0x90] = R2

4.5 LDS - Load Direct

Mnemonic	LDS Rd, a
Format	I-Type (16-bit with address in low byte)
Operation	$Rd \leftarrow (a)$
Description	Loads Rd from absolute memory address a. The address is an 8-bit immediate constant, allowing access to any location in 256-byte data RAM.
Flags Affected	None
Cycles	2
Example	LDS R1, 0x30 ; R1 = memory[0x30]

4.6 STS - Store Direct

Mnemonic	STS a, Rd
Operation	$(a) \leftarrow Rd$
Description	Stores Rd to absolute memory address a. The instruction encoding shows Rd in register field, address in immediate field.
Flags Affected	None
Example	STS 0x40, R2 ; memory[0x40] = R2

5 BRANCH AND JUMP INSTRUCTIONS

5.1 RJMP - Relative Jump

Mnemonic	RJMP k
Format	J-Type
Encoding	[Opcode][11-bit signed offset]
Operation	$PC \leftarrow PC + k + 1$
Description	Unconditional relative jump. Adds signed offset k to current PC+1. Offset range is -1024 to +1023 instructions (11-bit signed). The +1 accounts for PC already pointing to next instruction during execute phase.
Flags Affected	None
Cycles	2 (1 extra for PC update)
Example	RJMP 10 ; Jump forward 10 instructions RJMP -5 ; Jump back 5 instructions LABEL: ; Some label RJMP LABEL ; Infinite loop

5.2 RCALL - Relative Call

Mnemonic	RCALL k
Operation	$Stack \leftarrow PC + 1, PC \leftarrow PC + k + 1$
Description	Calls subroutine at relative address. Pushes return address (PC+1) onto stack, then jumps to subroutine. Uses same offset calculation as RJMP. Requires software stack management.
Flags Affected	None
Cycles	3 (push + jump)
Example	RCALL DELAY ; Call delay subroutine

5.3 RET - Return from Subroutine

Mnemonic	RET
Operation	$PC \leftarrow Stack$
Description	Returns from subroutine by popping return address from stack into PC. Must be paired with RCALL. Stack pointer management is manual.
Flags Affected	None
Cycles	4 (pop from stack)
Example	DELAY: ; ... subroutine code ... RET ; Return to caller

5.4 BRBS - Branch if Status Flag Set

Mnemonic	BRBS s, k
Format	Special I-Type: [Opcode][s-bit][8-bit offset]
Operation	If $SREG[s] = 1$ then $PC \leftarrow PC + k + 1$
Description	Conditional relative branch if specified status flag is set (1). The s parameter is flag index (0=C, 1=Z, 2=N, 3=V, 4=S, 5=H, 6=T, 7=I). Offset k is 8-bit signed (-128 to +127 instructions).
Flags Affected	None
Cycles	1 if no branch, 2 if branch taken
Example	BRBS 1, LABEL ; Branch if Z=1 (zero) BRBS 0, ERROR ; Branch if C=1 (carry)

5.5 BRBC - Branch if Status Flag Clear

Mnemonic	BRBC s, k
Operation	If $SREG[s] = 0$ then $PC \leftarrow PC + k + 1$
Description	Conditional relative branch if specified status flag is clear (0). Complement of BRBS.
Flags Affected	None
Example	BRBC 1, NOT_ZERO ; Branch if Z=0 BRBC 3, NO_OVF ; Branch if V=0

6 BIT AND I/O OPERATIONS

6.1 SBI - Set Bit in I/O Register

Mnemonic	SBI P, b
Format	I-Type variant
Operation	$I/O[P].b \leftarrow 1$
Description	Sets bit b (0-7) in I/O register at address P (0-31). I/O space is separate from data memory, typically mapped to GPIO control registers. This is atomic - no read-modify-write.
Flags Affected	None
Cycles	1
Example	SBI 0x05, 3 ; Set bit 3 of I/O register 5

6.2 CBI - Clear Bit in I/O Register

Mnemonic	CBI P, b
Operation	$I/O[P].b \leftarrow 0$
Description	Clears bit b in I/O register P. Atomic operation.
Flags Affected	None
Example	CBI 0x05, 3 ; Clear bit 3 of I/O register 5

6.3 BST - Bit Store to T Flag

Mnemonic	BST Rd, b
Operation	$T \leftarrow Rd.b$
Description	Copies bit b (0-7) from register Rd to T (Temp) flag in SREG. Used with BLD to manipulate individual bits without affecting other bits in register.
Flags Affected	T: Set to value of Rd[b]
Example	BST R3, 5 ; Copy bit 5 of R3 to T flag

6.4 BLD - Bit Load from T Flag

Mnemonic	BLD Rd, b
Operation	$Rd.b \leftarrow T$
Description	Copies T flag to bit b of register Rd. Only the specified bit is changed; other bits remain unchanged.
Flags Affected	None
Example	BLD R2, 0 ; Set bit 0 of R2 to T value

7 COMPARISON INSTRUCTIONS

7.1 CP - Compare Registers

Mnemonic	CP Rd, Rs
Operation	$Rd - Rs$ (sets flags only, Rd unchanged)
Description	Compares two registers by subtracting Rs from Rd, but doesn't store result. Only flags are updated. Used before conditional branches. CP R1,R2 sets flags as if R1-R2 was calculated.
Flags Affected	Same as SUB (Z, C, N, V, H)
Example	CP R3, R4 ; Compare R3 with R4 BRBS 1, EQUAL ; Branch if equal (Z=1)

7.2 CPI - Compare with Immediate

Mnemonic	CPI Rd, k
Operation	$Rd - k$ (sets flags only)
Description	Compares register with immediate constant. Subtracts k from Rd but preserves Rd.
Flags Affected	Same as SUB
Example	CPI R2, 10 ; Compare R2 with 10 BRBC 1, NOT_TEN ; Branch if not equal

8 INSTRUCTION SET SUMMARY

Mnemonic	Operands	Cycles	Brief Description
Arithmetic Operations			
ADD	Rd, Rs	1	$Rd = Rd + Rs$
ADC	Rd, Rs	1	$Rd = Rd + Rs + C$
SUB	Rd, Rs	1	$Rd = Rd - Rs$
SBC	Rd, Rs	1	$Rd = Rd - Rs - C$
COM	Rd	1	$Rd = \text{NOT } Rd$ (ones complement)
NEG	Rd	1	$Rd = 0 - Rd$ (twos complement)
Logical Operations			
AND	Rd, Rs	1	$Rd = Rd \text{ AND } Rs$
OR	Rd, Rs	1	$Rd = Rd \text{ OR } Rs$
XOR	Rd, Rs	1	$Rd = Rd \text{ XOR } Rs$
Data Transfer			
LDI	Rd, k	1	$Rd = k$ (immediate load)
MOV	Rd, Rs	1	$Rd = Rs$ (register copy)
LD	Rd, X	2	$Rd = \text{memory}[R7]$ (indirect load)
ST	X, Rs	2	$\text{memory}[R7] = Rs$ (indirect store)
LDS	Rd, a	2	$Rd = \text{memory}[a]$ (direct load)
STS	a, Rd	2	$\text{memory}[a] = Rd$ (direct store)
Branch/Jump			
RJMP	k	2	$PC = PC + k + 1$ (relative jump)
RCALL	k	3	Call subroutine at $PC+k+1$
RET		4	Return from subroutine
BRBS	s, k	1/2	Branch if status flag $s = 1$
BRBC	s, k	1/2	Branch if status flag $s = 0$
Bit Operations			
SBI	P, b	1	Set I/O bit
CBI	P, b	1	Clear I/O bit
BST	Rd, b	1	$T = \text{bit } b \text{ of } Rd$
BLD	Rd, b	1	$\text{bit } b \text{ of } Rd = T$
Comparison			
CP	Rd, Rs	1	$Rd - Rs$ (set flags)
CPI	Rd, k	1	$Rd - k$ (set flags)

Table 2: Complete Instruction Set Summary